SCIENCE AND TECHNOLOGY COUNCIL
IIT(BHU) VARANASI

INTER IIT TECH MEET 15.0

PREPATHON PS

NATURAL LANGUAGE PROCESSING

DCs: Tanishq Ahuja (8553505350)

SCIENCE AND TECHNOLOGY COUNCIL
IIT(BHU) VARANASI

Agentic Customer 360 — Proactive Intervention Desk

1. PROBLEM STATEMENT DESCRIPTION

1.1 The Domain and The Status Quo

"Customer 360" has long been the holy grail of enterprise architecture: the goal of aggregating a single, unified view of a customer's entire footprint across a business. Historically, large organizations have attempted to solve this by dumping data into massive lakes and building complex CRM dashboards. However, a fundamental problem remains: unifying data does not equate to acting on it. Dashboards are passive. They rely on a human account manager or support rep to log in, synthesize the information, and make a judgment call. When dealing with thousands or millions of customers, it is impossible for humans to continuously monitor these unified profiles to catch subtle, compounding shifts in behavior — like a high-value client quietly building up churn risk, or a missed window for a strategic upsell.

1.2 The Agentic AI Solution and The Data Influx

Agentic AI offers a paradigm shift from passive observation to continuous, autonomous reasoning. Modern customers generate a relentless, asynchronous influx of multi-modal data: support tickets, e-commerce transactions, travel histories, daily spends, product telemetry events, and communication transcripts. Instead of merely plotting these disconnected data points on a chart, an agentic system can continuously parse this chaotic stream, separating signal from noise, and synthesizing a coherent narrative of the customer's current state.

1.3 Ambient Agents vs. Prompt-Based Chatbots

Crucially, this solution is not a chatbot. Traditional generative AI relies on a prompt-based paradigm — it sits dormant until a human asks it a question. For Customer 360, we require **ambient agents**. These are living, background processes that do not wait for human initiation. They are triggered asynchronously by environmental changes: an event-driven trigger (e.g., a dropped transaction or a furious support ticket) or a timely scheduled occurrence (e.g., a 30-day account health check).

To achieve this, these ambient agents must utilize sophisticated memory architectures. By maintaining both short-term working memory (recent interactions) and long-term memory (historical baselines, past interventions, and life events), these agents build an evolving state for each customer. When a new event occurs, the ambient agent doesn't just evaluate it in a vacuum; it uses its memory to contextualize the event, proactively reacting to the environment to flag risks or initiate actions before a human even knows to look.

VERY IMPORTANT: This is a genuinely unfamiliar problem space for most teams — production-grade patterns for agent memory, MAS coordination, and streaming RAG freshness aren't things most people have built intuition for through everyday coding experience. That makes reading how others have approached these problems (papers, engineering blogs, conference talks) less of an optional nicety here and more of a load-bearing part of the work: the architectural choices later in this document will only hold up under real conditions if they're grounded in something more than guesswork.

2. NON-NEGOTIABLE KEY COMPONENTS

As Agentic AI transitions from experimental sandboxes to live, mission-critical enterprise environments, trust and safety become the primary bottlenecks to adoption. Unlike traditional deterministic software, agentic systems are inherently non-deterministic and can produce unpredictable, compounding sequences of actions. To safely deploy autonomous agents that interact with customer data and financial levers, the following components are absolute prerequisites for any upcoming agentic architecture:

Explainability: If an agent makes a decision, there must exist proper, reproducible reasons and clear decision paths that articulate how it arrived at that conclusion. Black-box outputs are unacceptable in enterprise environments; the system must be able to cite its sources and logic.

Situation

The Action Agent decides to offer a high-value customer a substantial 500 retention credit. Before approving the action, the human-in-the-loop requires context. The agent provides a clear explanation: it cites a 40% drop in platform login frequency over the last three weeks (flagged by the Usage Agent) combined with a highly negative sentiment score on a recent, unresolved support ticket (flagged by the Support Agent).

Traceability: Every single action taken by the agent must be meticulously logged. This includes all API requests, internal tool calls, database queries, and RAG retrievals. This transparent audit trail is vital for post-mortems and debugging; it allows engineers to see exactly how the agent interacted with the environment at any given millisecond, making it much easier to figure out where things went wrong if they do.

Situation

An agent inappropriately flags a healthy, long-term customer as a "High Churn Risk" and spams the account manager with urgent alerts. Through traceability, the engineering team reviews the logs and discovers that during a retrieval step, the agent fetched a legally dense contract termination notice for a different customer with a similar name, poisoning its context window. The detailed log instantly isolates the bug to the entity resolution layer rather than a flaw in the agent's core reasoning.

Guardrails: These are hard-coded, deterministic rules and bypasses that override the agent's autonomy in extreme or high-risk scenarios. Because LLMs can occasionally hallucinate or fail to grasp severe real-world nuance, guardrails act as the ultimate safety net, ensuring that catastrophic, non-compliant, or wildly expensive actions are physically impossible for the agent to execute.

Situation

A customer sends a furious, legally threatening email demanding account cancellation and a full refund. The Synthesis Agent might try to be "helpful" by attempting to negotiate or offering a standard 10% discount. A hard-coded guardrail — configured to trigger on high-risk keywords like "lawsuit," "legal action," or "attorney" — instantly bypasses the agent's standard logic, halts all autonomous outbound communication, and directly routes the ticket to a specialized legal escalation queue.

Human-in-the-Loop (HITL): No matter how well-reasoned an agent's output is, autonomy without a checkpoint is a liability the moment real money, real relationships, or real regulatory exposure is on the line. HITL is not a failsafe bolted on for optics — it's the mechanism that lets the system act with speed on low-stakes decisions while reserving judgment for a human wherever the cost of being wrong is high, the situation is ambiguous, or the action is irreversible. Just as important, HITL is where the system earns trust over time: every approval or override a human gives becomes a data point the system can eventually learn from, tightening or loosening its own autonomy boundaries.

Situation

The system detects a pattern consistent with an inferred major life event — a large lump-sum deposit, followed within days by a change of registered address and a spike in searches for home-loan products on the customer's app sessions. The Offer Agent proposes proactively reaching out with a pre-approved mortgage offer. Even though every individual signal is well-grounded, the action involves an unsolicited

financial product pitch tied to a sensitive personal inference (the system doesn't actually know if this is an inheritance, a divorce settlement, or a property sale) — so the message is routed to a relationship manager for approval before it ever reaches the customer, rather than being sent autonomously.

3. WHAT TO ARRIVE AT (THE END GOAL)

It's easy for a problem statement like this to drift into "build something that watches customers and surfaces insights" — and that's precisely the trap to avoid. A system that only monitors and flags is not gradeable, because there's no terminal, falsifiable output to check it against. **The system must be accountable for a specific, committed decision, not a general sense of awareness.**

Concretely, the end goal is: **given a customer's live, evolving signal set, the system must decide whether to trigger an intervention, and which one** — from a bounded set of real, costed actions such as:

- **No action** — signals don't cross a meaningful threshold; the system explicitly decides *not* to act, and that decision should be as traceable as an active one.
- **Proactive retention outreach** — a discount, fee waiver, or loyalty gesture, sized appropriately to the customer's value and the severity of the risk.
- **Relationship-manager escalation** — a human-initiated call or meeting, used when the situation is too nuanced or high-value for an automated message to be appropriate.
- **Personalized offer** — a pre-approved product (loan, higher credit line, investment product) surfaced at a moment inferred to be relevant, subject to eligibility rules.
- **Support/service intervention** — proactively reaching out to resolve a brewing issue before the customer has to escalate it themselves.
- **Compliance/fraud hold** — freezing or flagging an action pending human review, when the signal pattern looks like fraud, AML risk, or account takeover.

Alongside this bounded action set, there is a second, standing goal that runs underneath every decision the system makes: **the system must maintain an accurate, continuously updated understanding of what life event the customer is currently going through, inferred from the pattern of transactions and actions on their account** — a new child, a marriage, a promotion, a home purchase, financial distress, retirement, and so on. This inferred "current life phase" is not a one-off classification to be computed and forgotten; it belongs in the sophisticated memory architecture the participants design, alongside the customer's episodic history, and it should be the thing multiple agents *read from* rather than something each agent tries to re-derive independently every time. Concretely: a life-event inference made by the swarm/synthesis layer today should still be available and correctly weighted weeks later when the Offer Agent is deciding whether a personalized offer is relevant, or when the Retention Agent is deciding how to interpret a new signal in light of what's already known about this customer's situation. A system that infers "new parent" correctly on day one but doesn't carry that understanding forward — and instead pitches an unrelated generic offer three weeks later — has not actually met this goal, even if the initial inference was right.

A strong submission should be able to state, for any given moment in a customer's timeline: *what specific action (or explicit no-action) did the system decide on, why, and — where ground truth exists — was it the right call.*

4. EXPECTED ARCHITECTURE

4.1 Agent Memory

No single agent in this system should be a stateless function that evaluates one event in isolation — the whole premise of "ambient" agents is that they carry context forward. That means memory needs a deliberate design, not just a vector database bolted on as an afterthought.

Types of memory each agent can reason about: (not a strict requirement but more of an inspiration)

- **Working memory (short-term):** the live, evolving state of a single customer *right now* — the last few events, the current open case, what other agents have already concluded in this session. This is cheap to write and read, and gets discarded or compressed once the current situation resolves.
- **Episodic memory (long-term, per-customer):** a running history of what's happened to this specific customer over time and what was done about it — past interventions, past flags, past outcomes ("last time usage dropped like this, a proactive call prevented churn; a discount email was ignored"). This is what lets the system get smarter about *this* customer rather than treating every event as the first one.
- **Semantic / shared memory (cross-customer, cross-agent):** general patterns and policies that aren't tied to one customer — what a "typical" pre-churn usage pattern looks like across the whole book of customers, current offer eligibility rules, current compliance policy. This is closer to a shared knowledge base than a personal diary, and multiple agents read from it without needing to "own" it.

How memory is shared between agents: Rather than every agent maintaining its own private, siloed memory (which risks agents disagreeing on facts or duplicating retrieval work), the recommended pattern is a **shared per-customer state board** — a structured, continuously updated record that every agent can write findings to and read from, scoped strictly to one customer/account ID. An agent doesn't get raw access to another agent's internal reasoning trace, only to the structured conclusion it published to the board (e.g., the Usage Agent publishes `login_frequency_trend: -40%, confidence: high`, not its full chain-of-thought). This keeps handoffs clean and prevents context pollution, while still letting downstream agents (like a Synthesis Agent) build a coherent picture.

Open design questions participants must answer, not assume:

- When does an agent decide a past episodic memory is *relevant enough* to retrieve for the current situation, versus irrelevant noise from an unrelated past incident?
- How does the system prevent memory leakage or bleed-through between customers, especially at scale with thousands of concurrent per-customer memory stores?
- Who is responsible for memory decay/expiry — should a churn-risk flag from 18 months ago still influence a decision today?

4.2 Multi-Agent System (MAS) Design

Agents in this system don't all talk to each other in the same way at every stage — different points in the pipeline call for different coordination patterns. Participants should deliberately choose (and justify) which pattern fits which stage, rather than defaulting to one topology everywhere. This is exactly the kind of decision that should be informed by existing research and case studies on multi-agent coordination, not picked arbitrarily — there is real, documented experience out there on where each pattern tends to succeed or break down, and it's worth reading before committing to one.

- **Handoffs (sequential):** One agent finishes its work and passes a defined, scoped package of context to the next agent in the chain, without needing to loop back. *Best for:* linear pipelines where each stage is a genuinely separate concern — e.g., Usage Agent → Synthesis Agent → Action Agent. *Advantage:* simple, predictable, easy to trace. *Disadvantage:* no opportunity to catch an error introduced earlier in the chain; a bad handoff early on silently corrupts everything downstream.
- **Swarm (parallel, independent):** Multiple agents work on different slices of the same problem simultaneously and independently (e.g., Usage Agent, Support Agent, Transaction Agent all watching the same customer at once), writing their findings to the shared state board without waiting on each other. *Best for:* the initial signal-gathering stage, where different data sources genuinely don't depend on each other. *Advantage:* fast, scales horizontally. *Disadvantage:* someone downstream still has to reconcile disagreement between swarm members — the swarm itself doesn't resolve conflicting reads.

- **Round robin:** Agents take turns contributing to a shared draft or decision in a fixed sequence, each building on or revising the previous agent's contribution. *Best for:* iterative refinement tasks like drafting a personalized outreach message, where tone, factual grounding, and compliance checks each need a pass. *Advantage:* every concern gets a dedicated pass. *Disadvantage:* slower than parallel approaches; ordering matters and a bad early pass can bias later ones.
- **Critique-Refiner:** One agent proposes an output, a second (often with a different system prompt or persona) critiques it against a rubric or policy, and the first agent revises. *Best for:* anything going in front of a human or a customer — e.g., a proposed retention offer gets critiqued for cost-feasibility and compliance before it reaches HITL. *Advantage:* meaningfully reduces low-quality or non-compliant outputs reaching a human. *Disadvantage:* adds latency; not suitable for anything needing sub-second response.
- **Agent Debate:** Two or more agents holding genuinely different perspectives argue a case explicitly, with their disagreement made visible rather than silently averaged away. *Best for:* situations where reasonable agents could disagree and the disagreement itself is informative — e.g., a Churn-Risk Agent says "high risk, offer a discount," an Upsell Agent says "this customer is actually a growth opportunity, don't undersell them" — the debate output ("both are true — this is a customer to invest in, not discount") is more useful than either agent's answer alone. *Advantage:* surfaces genuine ambiguity instead of hiding it. *Disadvantage:* needs a defined resolution mechanism (who breaks the tie — another agent, a rule, or the human?), or it just produces an unresolved argument.

A realistic system will combine these: **swarm** for signal-gathering, **handoff** into a **synthesis** stage, **debate** when swarm agents disagree, **round robin** for drafting a customer-facing message, and **critique-refiner** as a final quality/compliance pass before anything reaches HITL.

4.3 Ambient Agents and Trigger Types

Every agent in this system is "ambient" — none of them wait for a human to open a chat window and ask a question. But not all triggers work the same way, and participants should be explicit about which type governs each agent:

- **Time-based (scheduled) triggers:** the agent runs on a fixed cadence regardless of events — e.g., a 30-day account health check, a quarterly KYC re-verification, a nightly rollup of the past day's transactions. These catch slow-building patterns that no single event would surface.
- **Event-based (reactive) triggers:** the agent fires the moment a specific event arrives in the stream — a new support ticket, a large transaction, a failed login. These are the fast-path triggers, and are what most of the "real-time" feel of the system depends on.
- **Agent-dependent (derived) triggers:** the agent fires not because of a raw external event, but because *another agent* published a finding that crosses some threshold — e.g., the Life-Event Inference Agent only activates once *both* the Transaction Agent (large deposit) and the Usage Agent (spike in home-loan related app searches) have independently flagged something in the same time window. This is what lets the system correlate weak signals into a strong one, instead of treating every raw event as independently actionable.

A well-designed system will use all three: time-based for baseline hygiene, event-based for responsiveness, and agent-dependent for the correlation logic that's the actual point of having multiple agents in the first place.

5. SUGGESTED AGENT ROSTER (STARTING POINT)

To make this problem statement approachable, here is a starting set of agents participants could build. This is a scaffold, not a fixed requirement — participants are encouraged to merge, split, or add agents as their design demands, as long as they can justify the change.

Agent	What It Monitors & Analyses	Toolset	Trigger Event
Usage/Engagement Agent	App/web session frequency, feature usage, login patterns, drop-off trends	Windowed aggregation queries over event stream, anomaly/trend detection model	Event-based (each usage event) + Time-based (daily rollup)
Support/Sentiment Agent	Support tickets, chat transcripts, call summaries, sentiment and urgency	Sentiment classification model, ticket history retrieval, keyword/urgency scanner	Event-based (new ticket or message)
Transaction/Billing Agent	Payments, transfers, billing history, spend patterns, large or unusual transactions	Transaction DB query, statistical anomaly scoring, historical baseline comparison	Event-based (real-time transaction stream)
KYC/Compliance Agent	KYC document status, sanctions/watchlist matches, regulatory flags	KYC database query, sanctions list lookup, compliance rules engine	Event-based (KYC update) + Time-based (periodic re-verification)
Life-Event Inference Agent	Cross-signal patterns suggesting a major life change (relocation, inheritance, new job, marriage)	Cross-table joins across Transaction/Usage/KYC findings, optional consented external enrichment	Agent-dependent (fires when two or more swarm agents flag correlated anomalies)
Synthesis/Correlation Agent	Reconciles all swarm findings into one coherent customer state; resolves contradictions	Shared state-board read/write, memory retrieval (episodic + semantic)	Agent-dependent (fires once swarm agents have published to the state board)
Offer/Eligibility Agent	Checks proposed actions against current product/offer eligibility rules and policy	Policy RAG retrieval, rules engine, eligibility calculator	Agent-dependent (fires after Synthesis Agent produces a candidate opportunity)
Retention/Action Agent	Decides and drafts the specific intervention: message, escalation, or offer	LLM message composer, RAG grounding against CRM/core records, episodic memory of past interventions	Agent-dependent (fires after Synthesis + Offer Agents agree there's a case)
Critique/Compliance-Refiner Agent	Reviews the proposed action for cost-feasibility, tone, and compliance before it reaches a human	Policy RAG, guardrail rule checker, cost estimator	Agent-dependent (fires immediately after Action Agent drafts a proposal)
Escalation/Guardrail Agent	Scans for hard-stop conditions (legal threats, fraud keywords, self-harm men-	Deterministic keyword/classifier scanner, direct routing rule	Event-based (fires instantly on pattern match, independent of the rest of the pipeline)

Agent	What It Monitors & Analyses	Toolset	Trigger Event
	tions) that must bypass the normal pipeline entirely		

6. PRODUCTION BAR CHECKLIST

A demo that works once, on a clean happy-path run, is not the same as a system built to a production bar. This checklist is what separates the two — participants should be able to point to each item below and show, not just claim, that it's handled.

6.1 Streaming Ingestion & Live Retrieval

- ☐ **Continuous ingestion pipeline** that accepts an ongoing, out-of-order, multi-source event stream (not a one-shot batch upload) — e.g., transactions, support tickets, app events, KYC updates all arriving asynchronously and at different velocities.
- ☐ **Transforms applied on the live stream**, not just at query time. For Customer 360, examples include: computing a rolling 30-day login-frequency average as new usage events arrive; joining a new transaction against the customer's historical spend baseline to compute a live anomaly score; enriching an incoming support ticket with a sentiment score the moment it lands; maintaining a windowed aggregation of transaction volume per account to catch a spike within seconds of it happening.
- ☐ **Vector DB / index kept live**, not rebuilt from scratch — when a new support ticket, policy document, or CRM note arrives, it should become retrievable without a full re-index, and stale embeddings should not silently persist past the point they're outdated.
- ☐ **Correct handling of late or out-of-order events** — a transaction or ticket that arrives out of sequence shouldn't corrupt a windowed aggregation or double-count in a rollup.

6.2 Multi-Agent Coordination

- ☐ **A working orchestration layer** — whether orchestrator-workers, swarm, or a hybrid — where agents genuinely specialize (different tools, different data access, different prompts) rather than being copies of the same generalist agent with a different label.
- ☐ **Explicit handoffs** where context passed from one agent to the next is deliberately scoped (structured findings, not a full context dump), and this scoping decision is documented and justified.
- ☐ **Demonstrated coordination under disagreement** — at least one scenario where two agents produce conflicting reads on the same customer, and the system has a defined mechanism (debate, critique-refiner, or an arbitration rule) to resolve it rather than silently picking one.

6.3 Human-in-the-Loop

- ☐ **A real approval checkpoint** before any customer-facing or costly action — not a log line claiming "human approved," but an actual interruption in the flow where a human can approve, reject, or modify the proposed action.
- ☐ **Escalation for ambiguity**, not just for cost — if the system's own confidence is low, or two agents disagree and can't resolve it, that should also route to a human, independent of the dollar amount involved.
- ☐ **An audit-ready approval log** — every HITL decision (approved, rejected, modified) is recorded with the context that was shown to the human at the time, so a reviewer can later reconstruct exactly what the human saw and decided.
- ☐ **A path for the human to ask "why"** — the human approver isn't just shown a recommendation, they can query the system for its reasoning and source citations before deciding.

6.4 Guardrails

- ☐ **Deterministic, hard-coded stop conditions** that cannot be reasoned around by the LLM — e.g., a legal-threat keyword match halts autonomous outbound communication regardless of what any agent "decides."
- ☐ **PII protection** — customer-identifying data (account numbers, SSNs/national IDs, full transaction details) is masked, tokenized, or redacted wherever it isn't strictly necessary for the current agent's task, including in logs and in any prompt sent to an LLM.
- ☐ **Role-based information access enforced at the data layer, not just the prompt layer** — an agent that isn't supposed to see trading/brokerage data, for instance, should be structurally unable to query it, not merely instructed not to. A prompt-level instruction ("don't look at X") is not a guardrail.
- ☐ **Safe tool-calling boundaries** — every tool an agent can invoke has a defined blast radius (e.g., an agent can *draft* a retention offer but cannot *send* it or *apply* a discount to the account without passing through HITL).

6.5 Observability, Traceability & Explainability

- ☐ **Full trace logging** — every agent invocation, tool call, retrieval, and handoff is logged with enough detail to reconstruct the full decision path after the fact.
- ☐ **Metrics, not just logs** — latency per stage (event time → decision time → action time), agent-level confidence scores, HITL approval/override rates, and false-positive/false-negative rates against any available ground truth.
- ☐ **Prompt and tool-call history retained per decision** — for any final action, it should be possible to pull up exactly which prompts were sent, which tools were called, and what each returned, not just the final summary.
- ☐ **Explainability as a first-class output, not a bolt-on** — every recommended or executed action ships with a citation-backed explanation (which signals, which memory, which source records) generated as part of the decision process itself, not reconstructed after the fact by asking the model to "explain what you just did."

7. DELIVERABLES

7.1 Research

A documented, running list of every paper, engineering blog, technical write-up, or video the participant consulted to understand this domain — with a short note per item on what was taken from it and which specific decision in the final system it informed. This isn't a bibliography for its own sake: it's evidence that architectural choices (which MAS pattern was used where, how memory was scoped, how retrieval freshness was handled) were arrived at deliberately, not defaulted into. Given how new and fragmented the practical literature on agentic systems still is, this domain genuinely rewards deep, deliberate reading over trial-and-error — the research log is where that has to show.

7.2 Codebase

- **AI usage:** Using AI tools to help implement your own idea, or to brainstorm and explore options, is not discouraged — that's a normal and expected part of how this kind of system gets built today. What *is* strongly discouraged is outsourcing the actual decisions to AI — which MAS topology to use, how to scope memory, what the trigger logic should be. Those calls should be the participant's own, informed by the research above; AI is a tool for execution and ideation here, not a substitute for the participant's own judgment.
- **Frontend expectations:** No flashy frontend is expected or rewarded. What is expected is a bare-minimum, working way to actually *see* the system doing its job — the influx of data arriving, agents acting on it, and the system arriving at (and explaining) its final decisions. A plain terminal log, a simple table view, or a minimal dashboard is entirely sufficient, as long as it's real and reflects what the system is actually doing, not a mocked-up illustration of it.
- **Integration:** All components — ingestion, agents, memory, retrieval, HITL, guardrails — must be stitched together and genuinely working end-to-end, not a collection of disconnected pieces that each work in isolation. A reviewer should be able to run the system and watch data go in one end and a decision come out the other.
- **Code structure:** The codebase should have clearly separated components for each agent, with each agent's prompt, tool definitions, and input/output schemas cleanly and explicitly structured — not buried inline or scattered across the codebase. A reviewer should be able to open the code and immediately identify what each agent is, what it can do, and what shape of data it expects and returns.

7.3 System Architecture Diagram

A cleanly drawn system architecture diagram that illustrates:

- Every component in the system — each agent, each data source, memory stores, the retrieval layer, HITL checkpoints, and guardrails.
- Where each component's data or input comes from, and what output it produces.
- Handoffs between agents shown explicitly and directionally — not a cloud of boxes with arrows everywhere, but a clear trace of "this agent hands this specific package of context to that agent."
- Each agent's toolset shown clearly, including what data source each tool reads from or writes to — a tool listed without a stated data source is an incomplete diagram.
- A visual distinction between synchronous flows (things that must happen immediately, like an authorization-time fraud check) and asynchronous flows (things that can happen on a schedule or in the background).

7.4 Solution Document (3 Pages)

A concise, 3-page document explaining the solution: the problem being solved, the architecture and why it was designed that way, the key decisions made (and the research or reasoning behind them), and known limitations or tradeoffs. Three pages is a hard constraint, not a suggestion — the ability to explain a complex agentic system clearly and concisely is itself part of what's being evaluated.

7.5 Testing & Evaluation

Results of testing and evaluation against the defined test scenarios — including what the system got right, what it got wrong, and where its reasoning or timing fell short. Participants should report this honestly rather than only showing favorable runs; a well-documented failure mode is worth more in evaluation than a suspiciously perfect scorecard.

8. EVALUATION CRITERIA

8.1 How Scenarios Will Be Evaluated

For each scenario, participants will be given the **live event stream** (replayed as described in the architecture sections above) — but they will *not* be given the hidden ground-truth answer key. What they will be given is a defined **inferred-events schema**: a required structure for how their system must log its own outputs over time — the inferred customer state at each point, the confidence level, and the action decided on (or explicit no-action), each timestamped.

It is the participants' responsibility to ensure their system actually writes out this inferred-events file in the specified schema as it processes the live stream — this is not an optional logging nicety, it is the artifact evaluators will score against the hidden ground truth. A system that reasons well internally but doesn't emit its conclusions in the required schema cannot be evaluated, and will be scored accordingly.

Brownie points will be awarded to participants that go a step further and build their own small scoring harness — a script or tool that takes their inferred-events output and a sample ground-truth file (from the practice scenarios, where this will be made available) and computes how well their system's inferences and actions line up with the known-correct answers. This is not required, but it demonstrates exactly the kind of rigor this problem statement is trying to encourage: treating the system's own output as something to be measured and validated, not just something to be demoed.

Criterion	Weight
Research	30%
Technical Implementation	25%
Architectural Novelty	20%
Non-Negotiable Components (Observability, Traceability, Guardrails, Explainability)	15%
Documentation and Architecture	10%

Research — 30%

This is the single heaviest-weighted criterion, deliberately. Agentic systems, streaming customer data, and multi-agent coordination are new enough as a practical discipline that intuition and prior general coding experience alone won't produce a good design — participants that read deeply about existing approaches to memory architecture, MAS coordination patterns, retrieval freshness, and HITL calibration will make visibly better decisions than participants that didn't. This is judged on the depth and relevance of what was actually read and *applied* — not the length of the reading list — as shown by the research log and by architectural decisions that are traceably grounded in it.

Technical Implementation — 25%

Does the system actually work, end-to-end, on real or realistically simulated streaming data? This covers correctness, robustness to messy or out-of-order data, and whether the components shown in the architecture diagram are genuinely present and functioning in the codebase — not just described on paper.

Architectural Novelty — 20%

Does the solution show original, well-reasoned thinking in how it composes memory, MAS topology, triggers, and retrieval — rather than defaulting to the most obvious single-agent-with-a-vector-database pattern? Novelty

here means a non-obvious design choice justified by the problem's actual constraints, not novelty for its own sake.

Non-Negotiable Components — 15%

Explicit, direct credit for whether observability, traceability, guardrails, and explainability are genuinely implemented and demonstrable in the running system — not merely claimed in the documentation. This is scored directly against the Production Bar Checklist above.

Documentation and Architecture — 10%

The clarity of the architecture diagram and the 3-page solution document: can a reviewer understand the system's design, data flow, and reasoning without having to reverse-engineer it from the code.

9. DEADLINES

Mid-Term Submission

- Preliminary research document
- Preliminary System architecture
- One page report about your findings in the domain of the PS and what approach you are going to follow to solve it.

You are allowed to edit and extend both your research and system architecture before the final submission. This is for the purpose of looking at your progress.

The mid-term submission must be done before **15th September EoD**.

End-Term Submission

The deliverables mentioned above are supposed to be compiled into a single GitHub repository which should be submitted on the google form circulated before **19th September EoD**.

Event stream link: https://mega.nz/file/9rh1iLwS#ek99IhEEXnOq2NW-PDukwpUxZnyAhTpGEFLC6-DSLkA